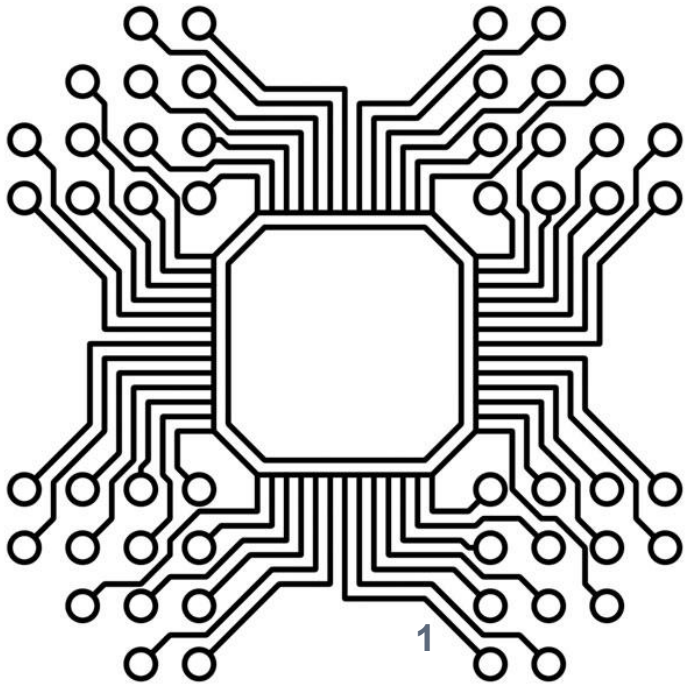
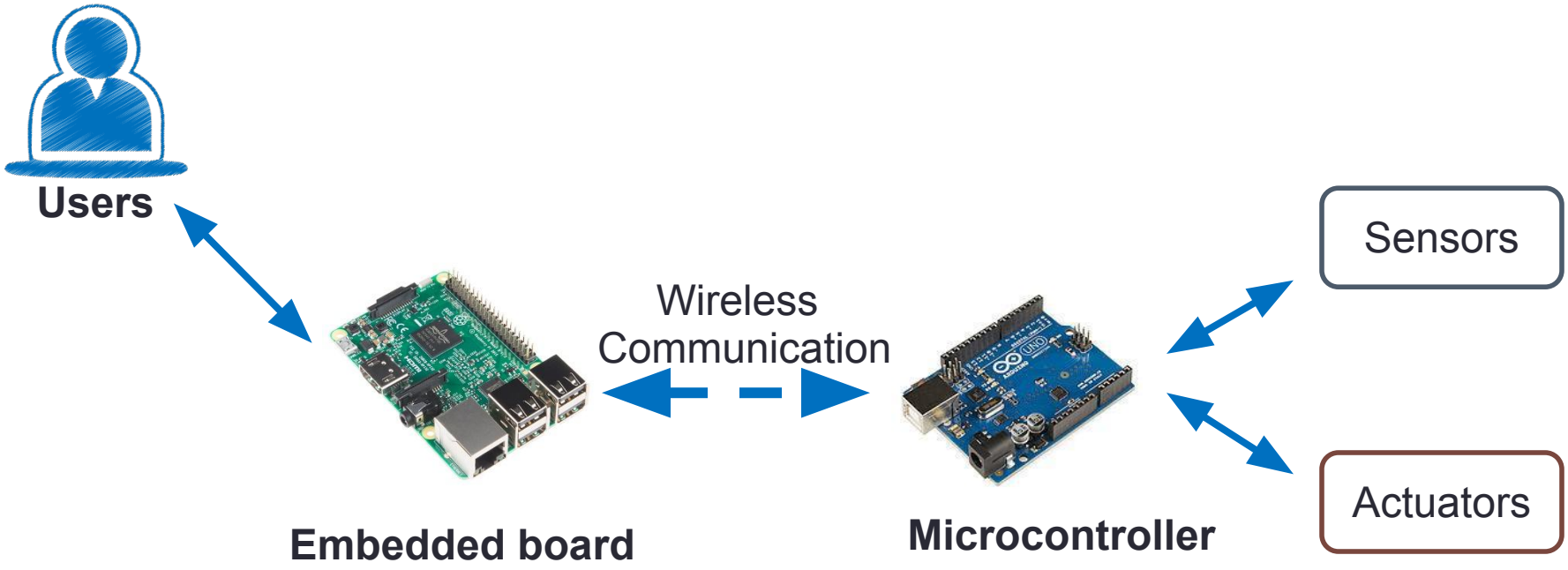
WEB SERVER

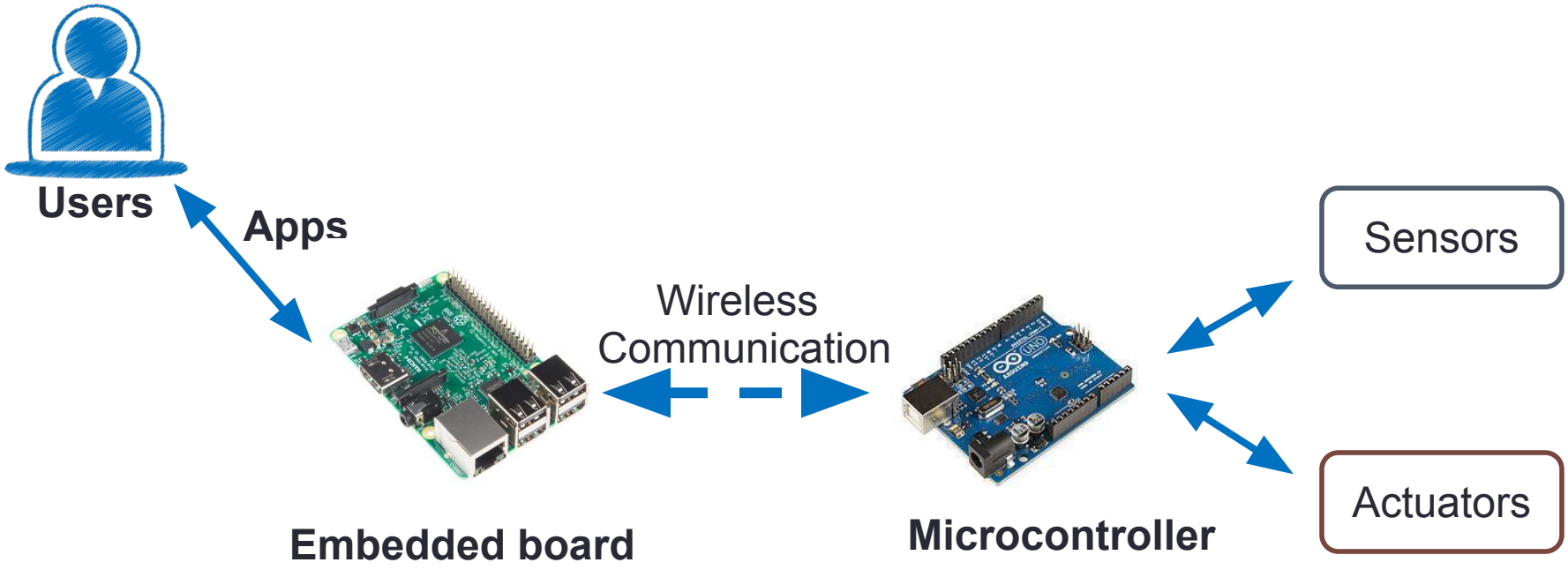
Lecturer: Dr. Bui Ha Duc
Dept. of Mechatronics
Email: ducbh@hcmute.edu.vn



System Diagram



System Diagram



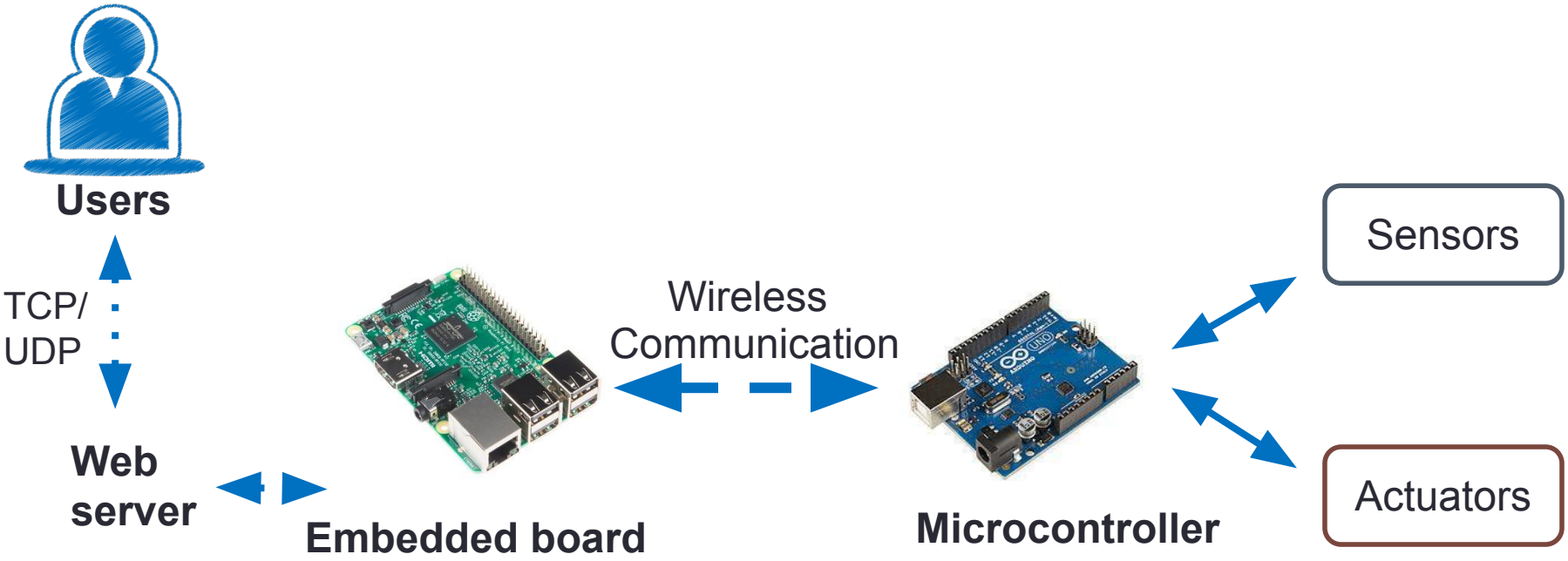
Advantages:

- Directly control the system
- Make use of the hardware capability

Disadvantages:

- Limited by the hardware and OS
- Can’t do remote control
- Difficult to access online database

System Diagram



Advantages:

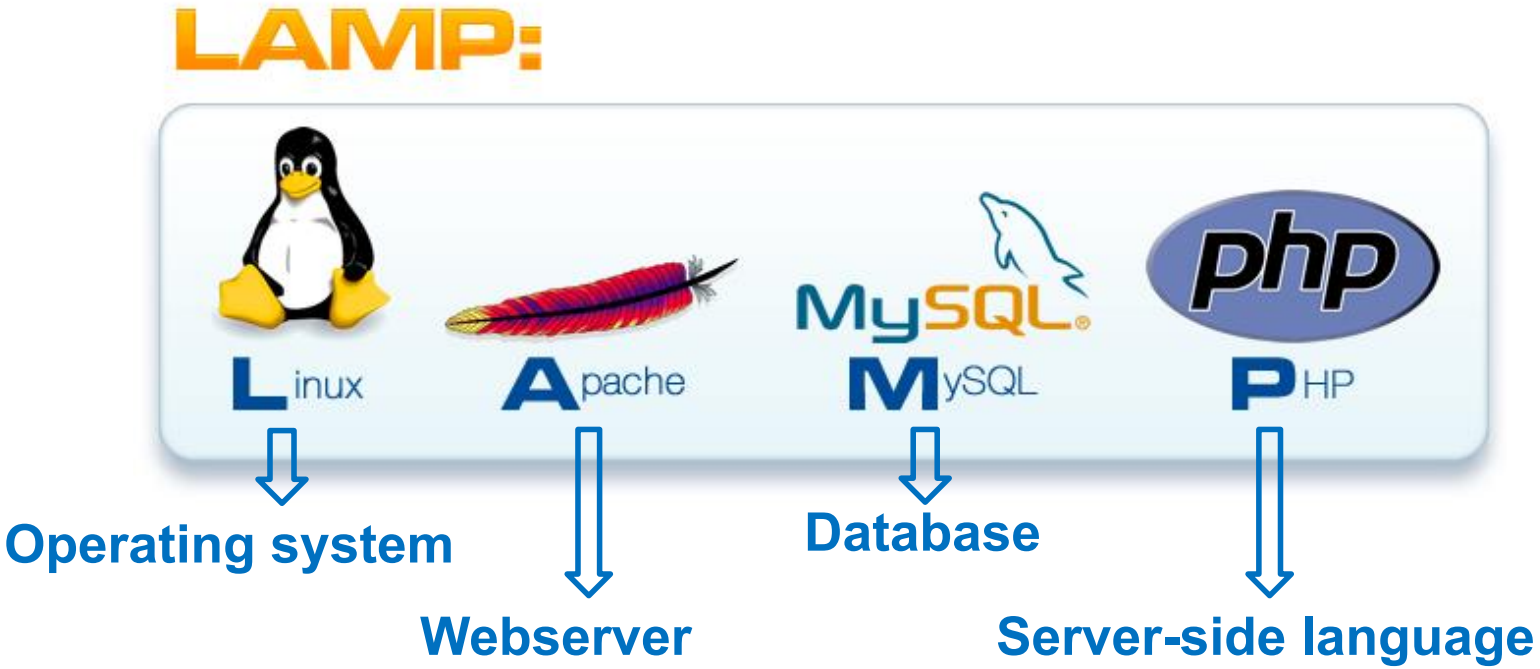
- Not depend on OS
- Can control/monitor the system from anywhere
- Work with resource available on the internet

Disadvantages:

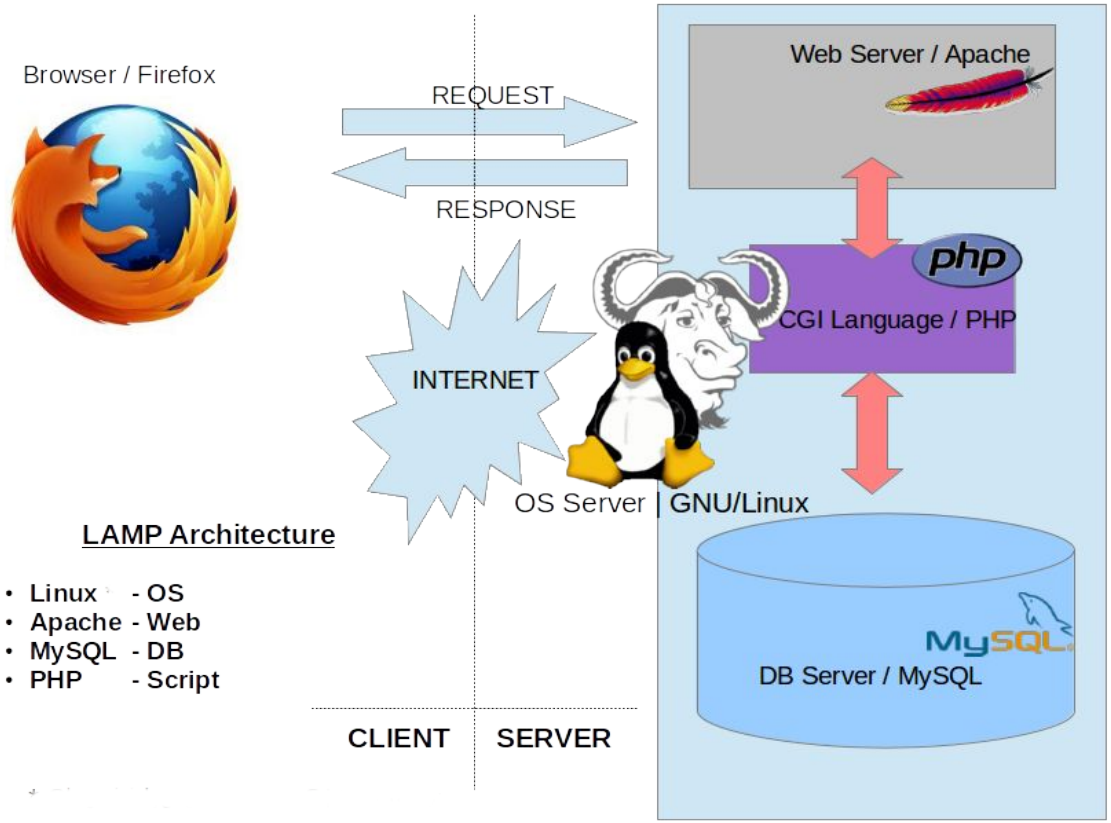
- Depend on wifi/internet
- Can't be very complex
  - Show data, plot graph
  - Simple algorithm/calculation

LAMP server

- LAMP is made up of Linux – Apache – MySQL – PHP



LAMP Server



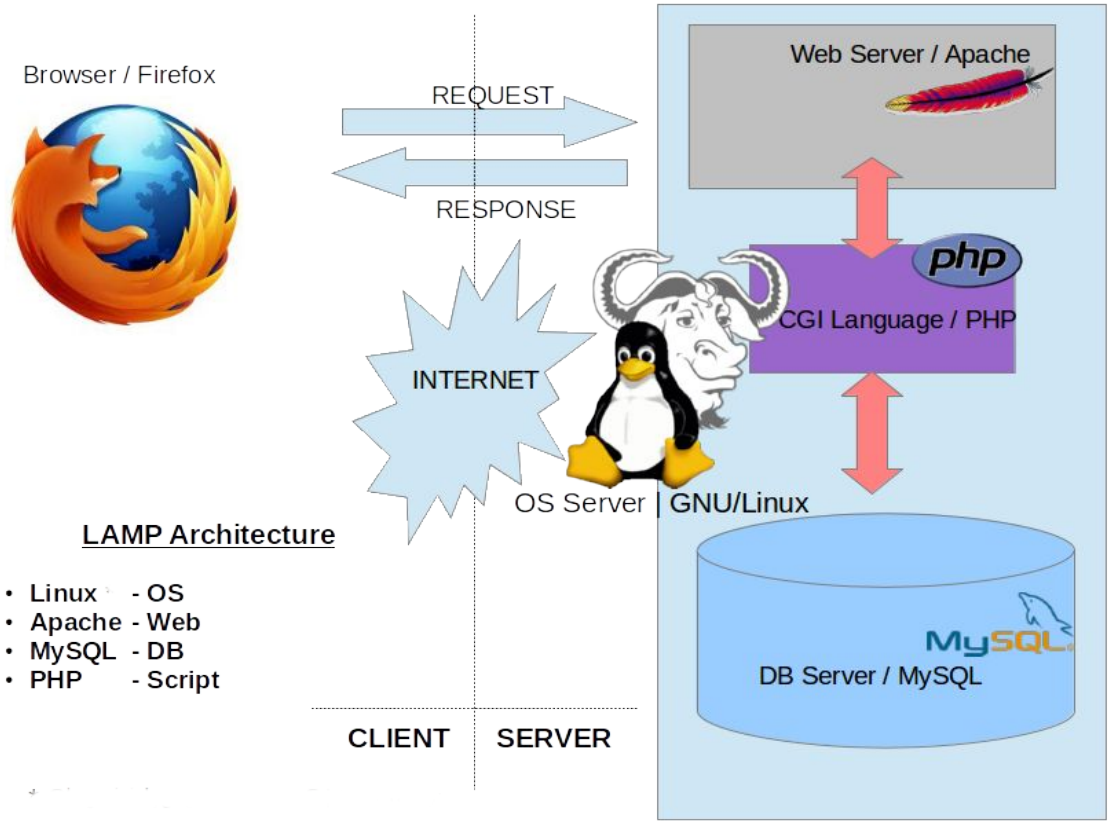
LAMP Architecture

- Linux - OS
- Apache - Web
- MySQL - DB
- PHP - Script



Hardware

LAMP Server



LAMP Architecture

- Linux - OS
- Apache - Web
- MySQL - DB
- PHP - Script



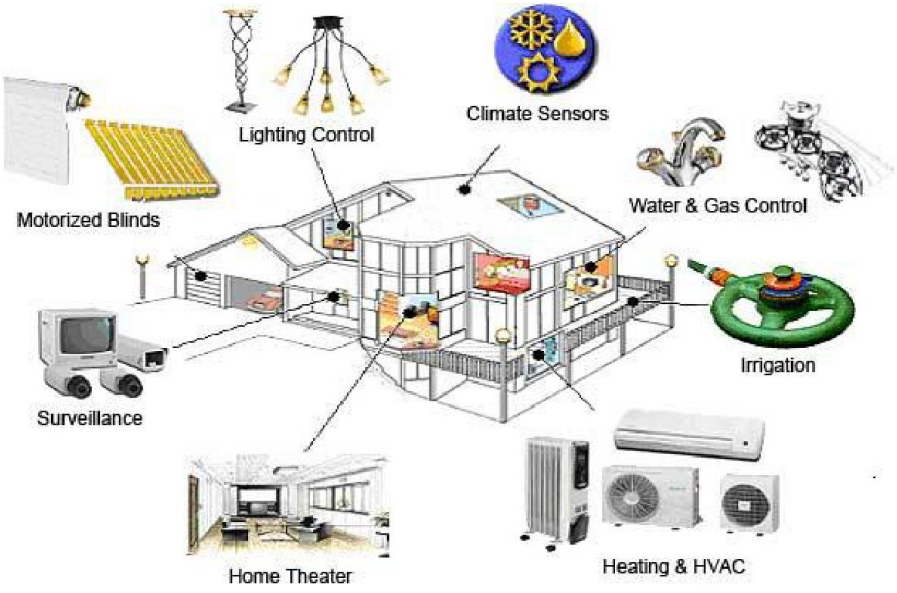
Hardware

Problems

- What is web server and its functions?
- How to create a web server?
- How to communicate / transfer data between the web server and the Friendly ARM?
- Why do we need an embedded system?

Web server

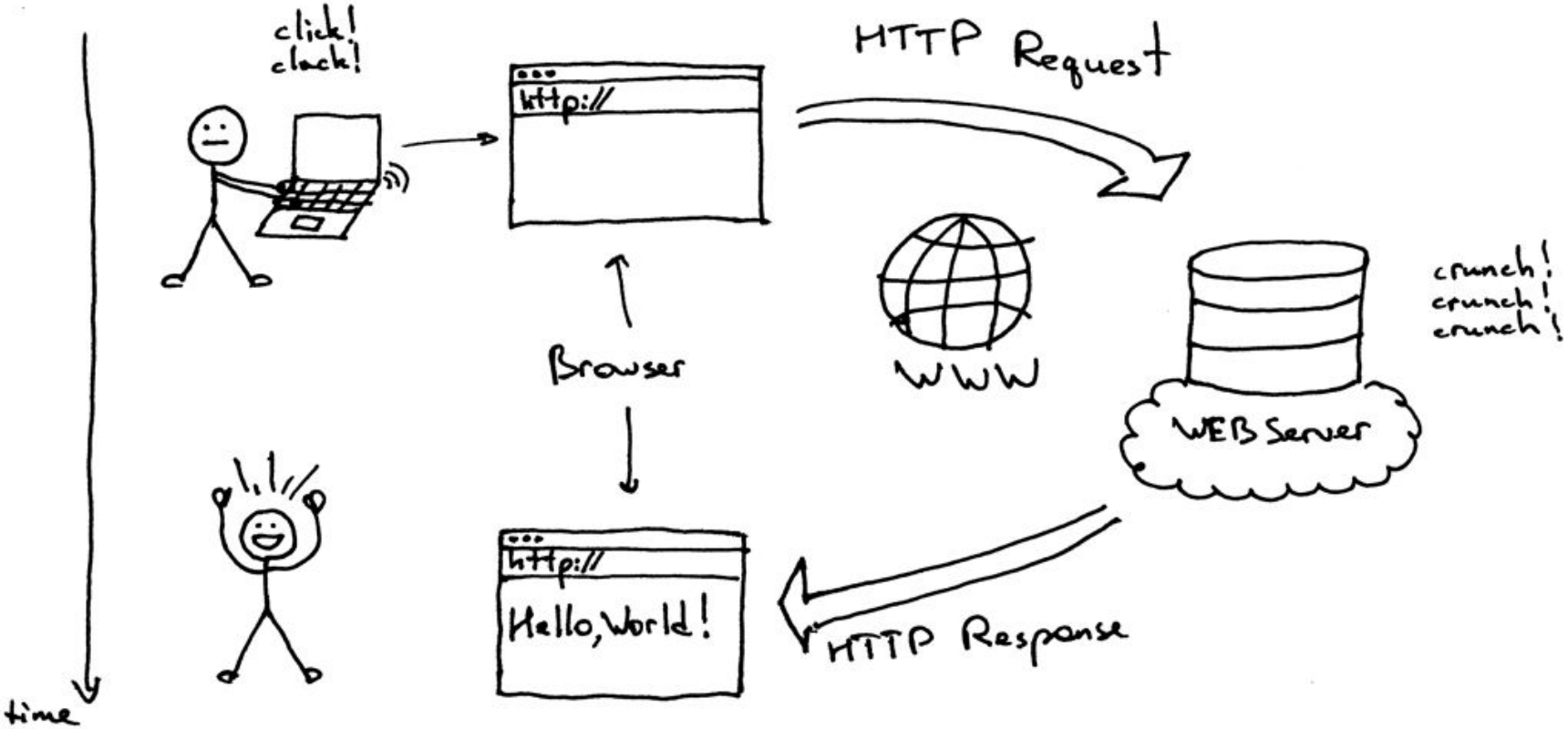
- Web server hosts a web site and provides reliable services for any requesting client.
- Web server communicate with clients via different protocols: TCP, UDP, IP…



Open sourced webservers for home automation:

- Calaos
- Domoticz
- OpenHAB
- OpenMotics
- Home assistant

Example of a web server



https://ruslanspivak.com/lsbaws-part1/

Creating a web server

Apache Webser:

- Most popular web server application
- Apache can serve HTML files over HTTP anddynamic web pages using scripting languages such as PHP javascript

Procedures:

1. Install apache with the command:

   ```
   sudo apt-get install apache2 -y
   ```

2. Find Raspberry IP with:

   ```
   hostname -I
   ```

3. Access the website at raspberry IP

Creating a web server

Changing the default web page

- The default website is located at:

  `/var/www/html/index.html`

- /var/www/html is a enclosed folder, owned by root.
- To modify, change the owner with:

  `sudo chown pi: index.html`

Or `sudo chown -R pi:pi /var/www/html`

- Replace the index.html with your website

Creating a website

Web developing Languages:

- **HTML**: creating Web pages
- **CSS**: create website layout, describes how HTML elements are to be displayed on screen, paper, or in other media
- **Javascript**: makes website do what you want them to do
- **PHP**: making dynamic and interactive Web pages
- **Websocket**: lightweight client-server communication

Visit **www.w3schools.com**

Structure of a website

```
<!DOCTYPE html>
<html>
<head>                          Elements
   <title>Page Title</title>
</head>
<body>

   <h1>My First Heading</h1>
   <p>My first paragraph.</p>

</body>
</html>
```

HTML Elements

- HTML element structure:

  <tagname>Content...</tagname>

  Start tag **End tag**

- HTML Elements

Start tag	End tag	Function
<h1>	</h1>	Heading text
		Paragraph
<a link>		Link address
<img>		Image
<table>	</table>	Table
		Division, contains other elements
<form>	</form>	Defines a form that is used to collect user input

HTML Attributes

- Attributes provide **additional information** about an element.
- Attributes are always specified in **the start tag**

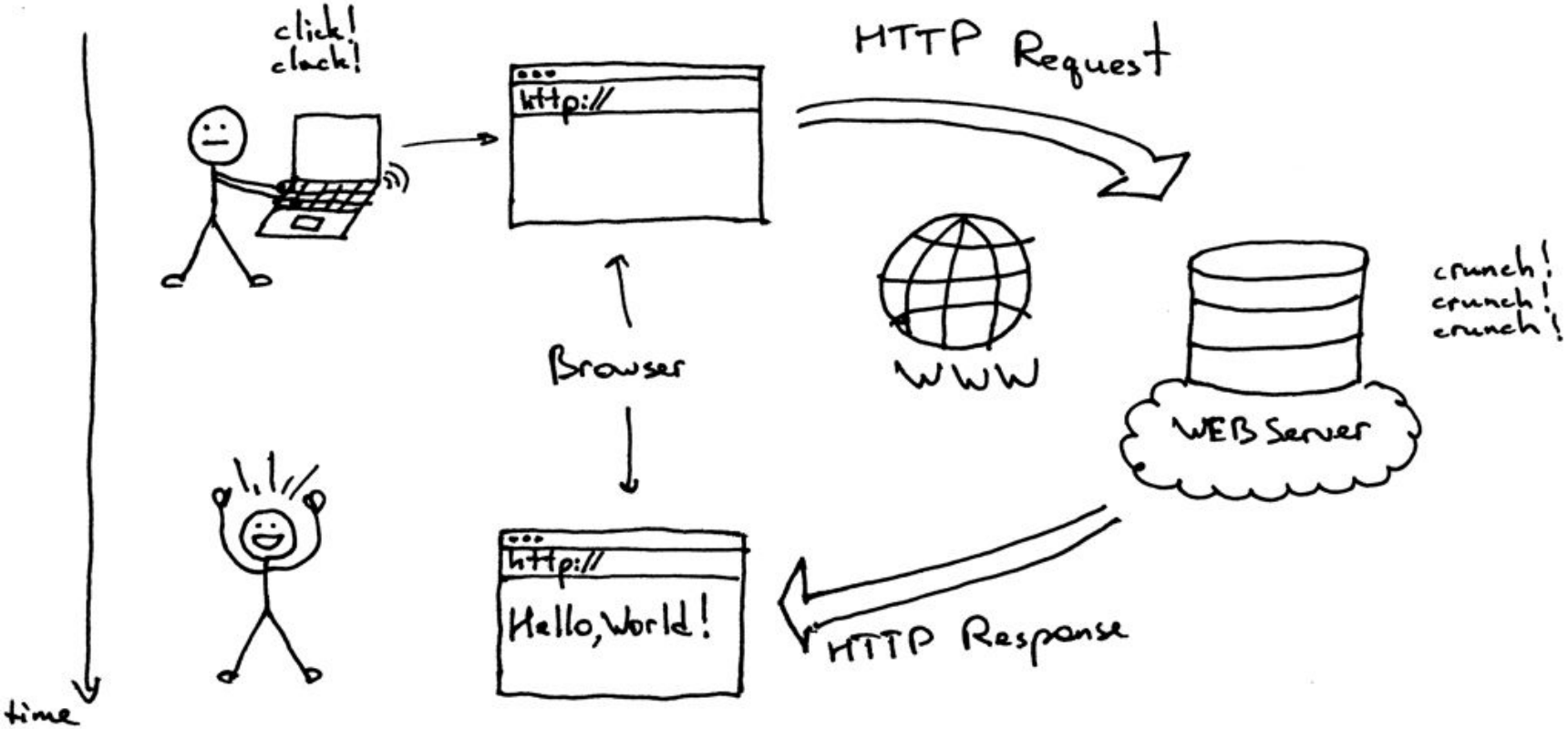
E.g. This is a link

<img src="w3schools.jpg" width="104" height="142">

- HTML Attributes:

Attribute	Function
`disabled`	Specifies that an input element should be disabled
`href`	Specifies the URL (web address) for a link
`id`	Specifies a unique id for an element
`src`	Specifies the source (web address) for an image
`style`	Specifies an inline CSS style for an element
`title`	Specifies extra information about an element

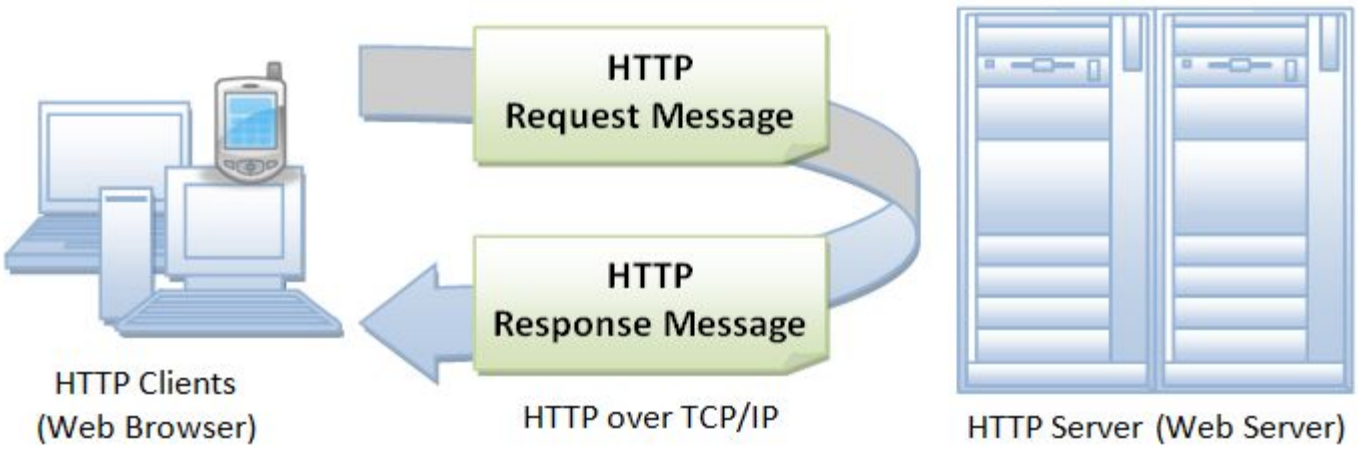
Example of a web server



https://ruslanspivak.com/lsbaws-part1/

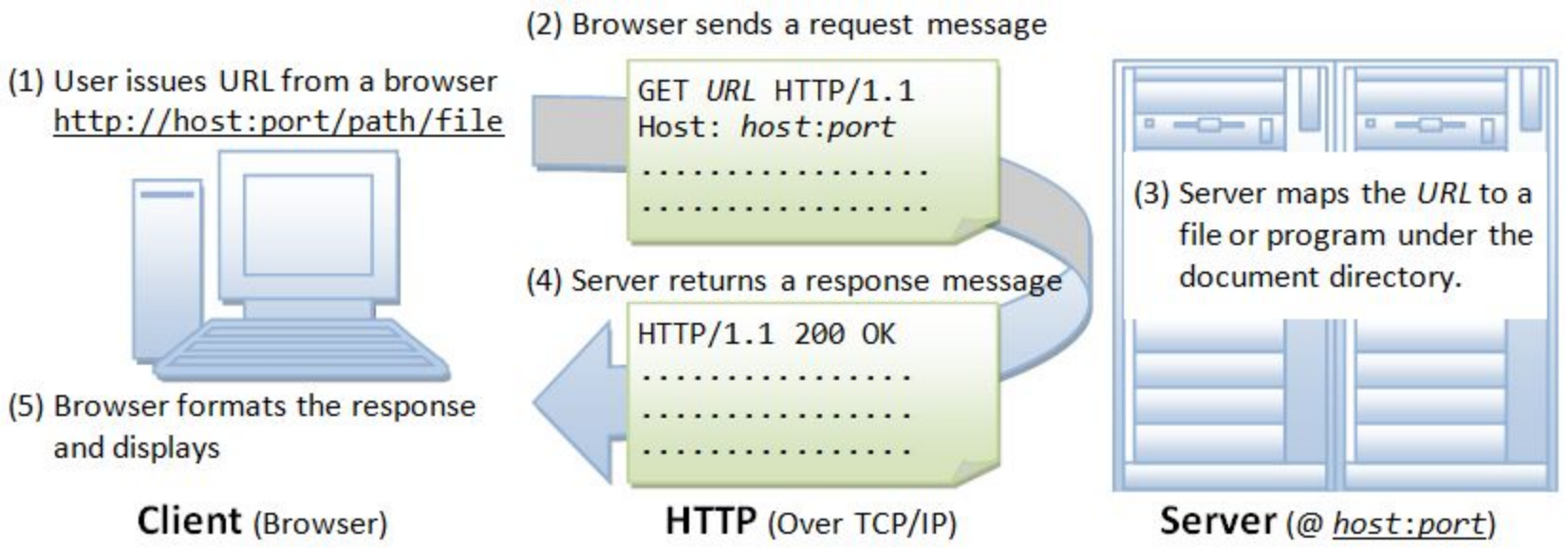
HyperText Transfer Protocol (HTTP)

- HTTP is an *asymmetric request-response client-server* protocol
- HTTP is a stateless protocol



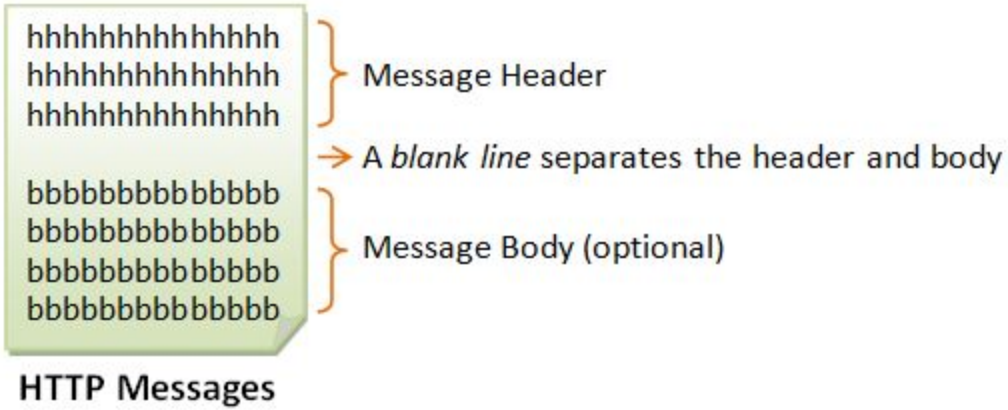
http://www.ntu.edu.sg/home/ehchua/programming/webprogramming/http_basics.html

HyperText Transfer Protocol

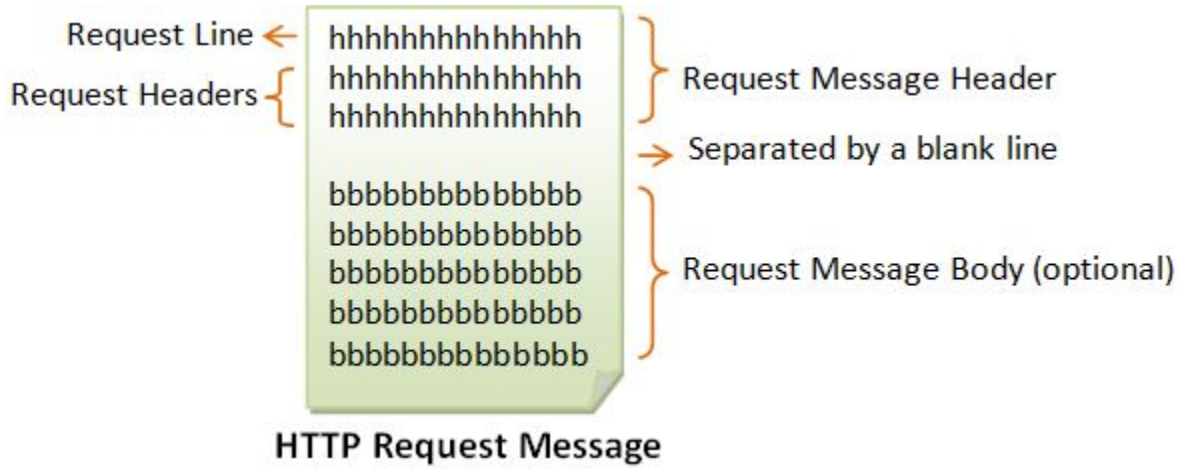


HTTP Messages

- HTTP client and server communicate by sending **text messages**
- An HTTP message consists of a *message header* and an optional *message body*, separated by a *blank line.*



HTTP Request Message



- The request line has the following syntax:

  request-method-name request-URI HTTP-version

  - *request-method-name*: request methods, e.g., GET, POST, HEAD, and OPTIONS.
  - *request-URI*: specifies the resource requested.
  - *HTTP-version*: Two versions are currently in use: HTTP/1.0 and HTTP/1.1.

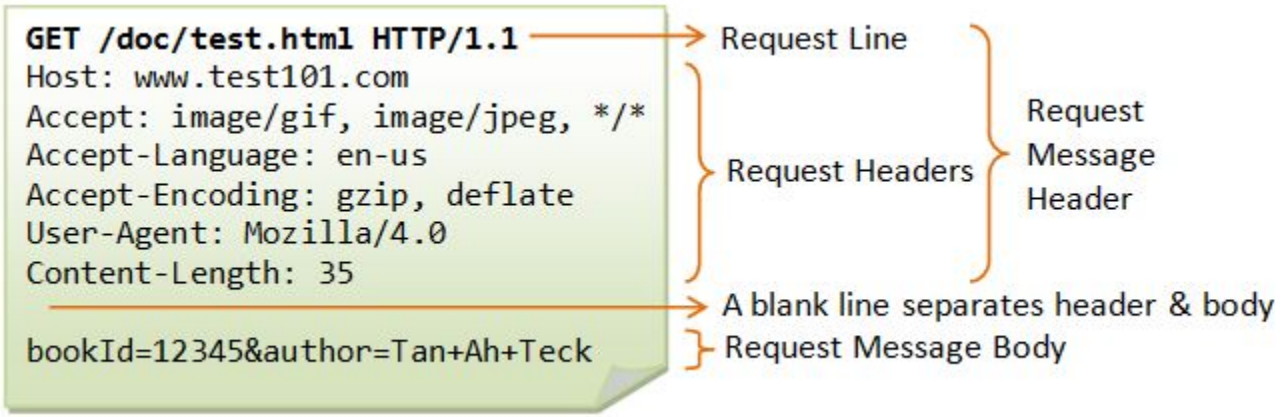
HTTP Request Message

- The request headers Syntax:

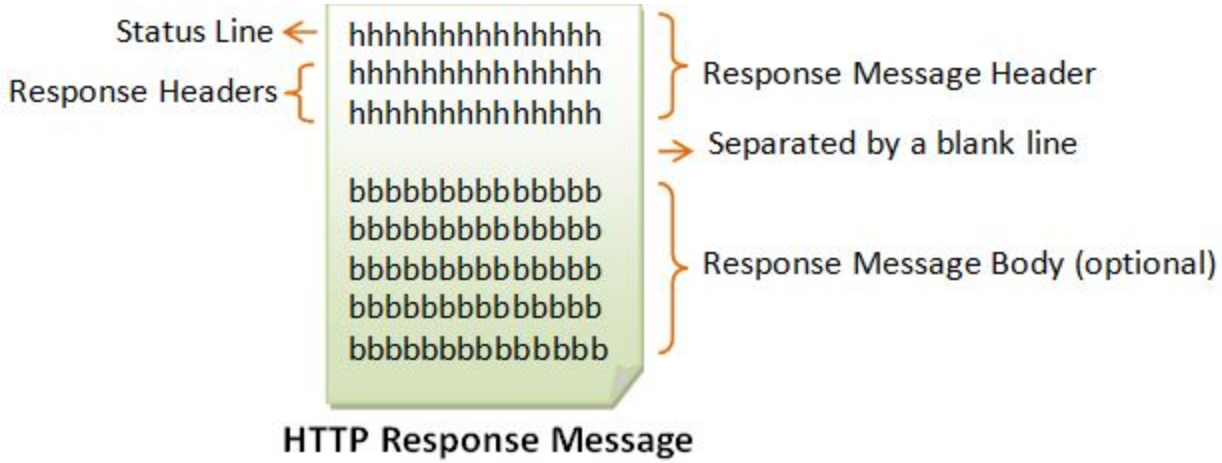
request-header-name: *request-header-value1*, *request-header-value2*, ...

- request message body: contain user inputs

  name1=value1&name2=value2...



HTTP Response Message



- The status line has the following syntax:

  HTTP-version status-code reason-phrase

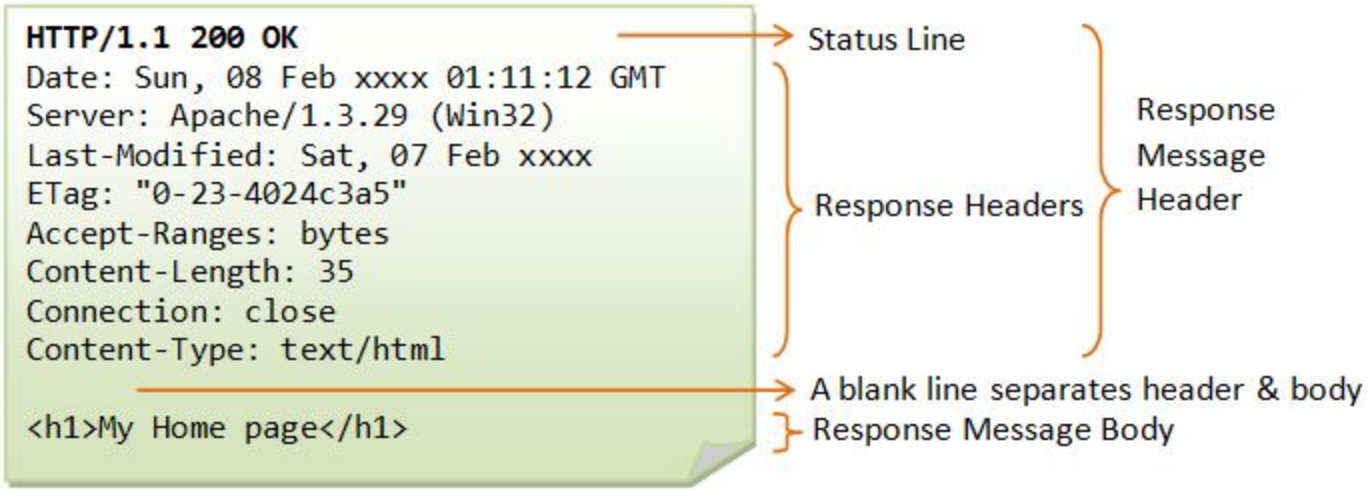
  - *HTTP-version*: The HTTP version used in this session. Either HTTP/1.0 and HTTP/1.1.
  - *status-code*: a 3-digit number generated by the server to reflect the outcome of the request. (200, 403, 404…)
  - *reason-phrase*: gives a short explanation to the status code

HTTP Response Message

- The response headers syntax:

```
response-header-name: response-header-value1, response-header-value2, ...
```

- Response message body: return value/ website



The Action Attribute

Example

```
<form action="…/mult.cgi">
```

- The **action** attribute defines the action to be performed when the form is submitted.

- In the example above, the form data is sent to a program on the server called "mult.cgi". This program contains a server-side script that handles the form data.

- If the action attribute is omitted, the action is set to the current page.

The Method Attribute

- The **method** attribute specifies the HTTP method (**GET** or **POST**) to be used when submitting the form data.

  e.g. <form action="…/mult.cgi" method = "GET">

  <form action="…/mult.cgi" method = "POST">

- If **GET** is used, the data received will be

  ?m=20&n=3

(data will be show on the url bar)

- **GET** is best suited for short, non-sensitive, amounts of data, because it has size limitations.

The Method Attribute

- If **POST** is used, the data received will be

  m=20&n=3

- **POST** has no size limitations, and can be used to send large amounts of data.

- *Reference*

(https://developer.mozilla.org/en-US/docs/Web/Guide/HTML/Forms/Sending_and_retrieving_form_data)

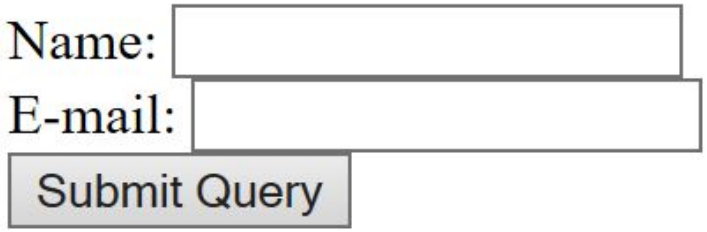
Reading input

- Using **POST**, data is passed to the script or program in the standard input stream (stdin), and the length (in bytes, i.e. characters) of the data is passed in an environment variable called CONTENT_LENGTH.

- When reading the input, the program must not try to read more than CONTENT_LENGTH characters.

(Refer to these website for more detail:
https://www.w3schools.com/php/php_forms.asp
https://www.ostraining.com/blog/coding/retrieve-html-form-data-with-php/
)

Reading input from POST




```
<!DOCTYPE HTML>
<html>
<body>
    <form action="welcome.php" method="post">
            Name: <input type="text" name="name"><br>
            E-mail: <input type="text" name="email"><br>
            <input type="submit">
    </form>
</body>
</html>
```

The data received will be ?

Reading input from POST with PHP

```
// welcome.php
<?php
// retrieve the form data by using the element's name attributes value as key
$name = $_POST[“name];
$email = $_POST[“email”];
// display the results
echo '<h3>Your name is</h3>’ .$name;
echo “<br>”;
echo '<h3>Your email is</h3>’ .$email;

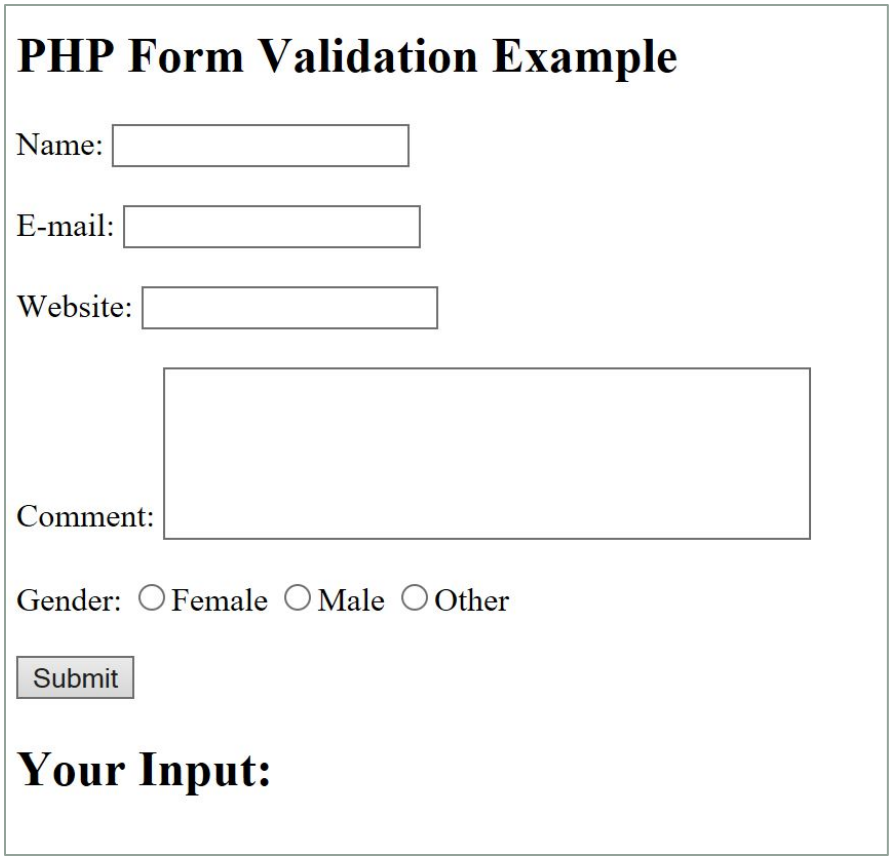
?>
```

Example

- How does this website look like?

```
<h2>PHP Form Validation Example</h2>
<form method="post" action="form.php">
  Name: <input type="text" name="name">
  <br><br>
  E-mail: <input type="text" name="email">
  <br><br>
  Website: <input type="text" name="website">
  <br><br>
  Comment: <textarea name="comment" rows="5" cols="40"></textarea>
  <br><br>
  Gender:
  <input type="radio" name="gender" value="female">Female
  <input type="radio" name="gender" value="male">Male
  <input type="radio" name="gender" value="other">Other
  <br><br>
  <input type="submit" name="submit" value="Submit">
</form>
```

Example



The data received will be ?

Write a php file to read the user input?

```php
<?php
// define variables and set to empty values
$name = $email = $gender = $comment = $website = "";

if ( isset( $_POST['submit'] ) )  {
  $name = $_POST["name"];
  $email = $_POST["email"];
  $website = $_POST["website"];
  $comment = $_POST["comment"];
  $gender = $_POST["gender"];
}

?>
```